

JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 16 OF 18

# Greedy Algorithms: Recognition, Proofs, and Scheduling

Local Choices, Exchange Arguments, Counterexamples, and Interval Decisions

---

FOUNDING AUTHOR

**Vinay Reddy Kalluri**

EDITOR-IN-CHIEF | CHIEF AUDITOR

Recognize when a local choice can be proved safe, construct counterexamples when it cannot, and implement sorting-driven greedy schedules.

---

GREEDY CHOICE | EXCHANGE PROOF | INTERVALS | SORTING | SCHEDULING

---

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

# Start Here - Your Route Through This Volume

**60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to 15 – Graphs. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Greedy solutions are short to code and hard to justify. This volume centers the proof obligation, not the superficial loop.

## Readiness map

| Decision      | Evidence                                                                                                                                                                                                           |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| START HERE    | A locally best choice may leave a compatible subproblem; Items can be ordered by a decisive endpoint or cost; The interviewer asks why dynamic programming is unnecessary; A tempting rule needs a counterexample. |
| PREPARE FIRST | Volumes 1-15; Sorting, comparators, graphs, and complexity.                                                                                                                                                        |
| MOVE ON WHEN  | State a greedy-choice property; Use exchange and stays-ahead reasoning; Implement interval scheduling safely; Distinguish proven greedy logic from unsupported intuition.                                          |

## Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

|             |                                                      |                          |
|-------------|------------------------------------------------------|--------------------------|
| 15 - Graphs | <b>YOU ARE HERE</b><br><b>16 - Greedy Algorithms</b> | 17 - Dynamic Programming |
|-------------|------------------------------------------------------|--------------------------|

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

| CONTENTS NAVIGATION                                                               |    |
|-----------------------------------------------------------------------------------|----|
| Start Here - Your Route Through This Volume                                       | 2  |
| Part I - Core Learning                                                            | 4  |
| Chapter 1 - SDE-2 Greedy Algorithms: Recognition, Proof, and Boundaries . . . . . | 4  |
| Part II - Practice and Handoff                                                    | 24 |
| Practice Ladder and Next Steps                                                    | 24 |

**Part I - Core Learning****SDE-2 CORE CHAPTER**

# Chapter 1 - SDE-2 Greedy Algorithms: Recognition, Proof, and Boundaries

## WHY IT WORKS | Why greedy needs a proof

A greedy algorithm commits to a locally preferred choice and does not revisit it. Sometimes that produces an optimal global solution; sometimes it produces a plausible wrong answer. The difference is not confidence or the number of examples tested. It is a structural proof showing that the local choice can belong to an optimum and that the remaining problem has the same form.

At SDE-2 level, name the choice rule, feasibility invariant, and proof method. If the proof fails, keep the greedy idea as a heuristic or switch to dynamic programming, search, or another exact algorithm. This chapter develops exchange, stays-ahead, and cut arguments across interval selection, merging, reachability, gas stations, scheduling, Huffman coding, fractional knapsack, and Dijkstra's boundary.

## Learning objectives

After completing this chapter, you should be able to:

- recognize when local choice and optimal substructure may support a greedy solution;
- distinguish feasibility from optimality;
- prove a greedy choice with exchange, stays-ahead, or cut arguments;
- maximize compatible activities by earliest finishing time;
- merge interval coverage after sorting;
- solve Jump Game reachability and minimum jumps by frontier invariants;
- derive the gas-station restart rule and global feasibility test;
- choose scheduling orders for specific objectives rather than one universal rule;
- build optimal Huffman merge cost and fractional-knapsack value;
- explain why Dijkstra is greedy only under nonnegative weights; and
- produce counterexamples and a disciplined greedy-versus-DP decision.

## Recognition checklist

A greedy approach is worth investigating when:

- the solution can be built as a sequence of irrevocable choices;
- choices have a natural ranking such as earliest finish, least cost, or best ratio;
- accepting one choice leaves a smaller problem of the same form;
- a locally chosen solution can be exchanged with the first choice of an optimum;
- one greedy partial solution can be shown to stay at least as good as any competitor; or
- a cut/property identifies one safe boundary-crossing choice.

Warning signals include:

- a choice consumes capacity and future value combinations matter;
- local scores do not account for interactions between choices;
- negative weights invalidate monotone progress;
- feasibility itself requires exploring alternatives; or
- a small counterexample defeats the proposed ordering.

Greedy is not "take the largest" or "sort and scan." The ranking must match the objective and admit a proof.

### WHY IT WORKS | The proof toolkit

#### Exchange argument

Take an optimal solution **OPT**. Show that its first choice can be replaced by the greedy choice without making the solution infeasible or worse. After the exchange, some optimum begins greedily. Apply the argument to the remaining subproblem.

This proves activity selection: replacing the first activity of an optimum with the compatible activity that finishes earliest leaves at least as much time for everything afterward.

#### Stays-ahead argument

Compare the greedy partial solution with any competitor after every number of choices. Prove a measure of greedy progress is never behind. If it stays ahead to termination, no competitor achieves a better objective.

This proves the Jump Game frontier algorithm: after considering every index in the current jump layer, greedy has the farthest boundary reachable with that number of jumps.

## Cut argument

Partition state into an accepted side and the remainder. Show that a best eligible item crossing the boundary is safe. Minimum spanning tree algorithms use the lightest cut edge. Dijkstra uses the smallest tentative-distance vertex, with nonnegative edges making it impossible for a later route across the cut to improve that distance.

## Greedy-choice plus induction

After proving one safe choice, show the residual problem has the same structure, then apply induction. The proof must cover both the first choice and the recursive shape; optimal substructure alone does not prove the greedy choice.

## Pattern 1: maximum compatible activities

Each activity is half-open interval  $[start, end)$ . Sort by increasing end time, then accept an activity when its start is at least the end of the last accepted activity.

Invariant: accepted activities are mutually compatible and, among schedules with the same accepted count drawn from the processed prefix, greedy's last finish time is no later than that of a competitor produced by exchanging choices.

## Exchange proof

Let  $g$  be the earliest-finishing activity and  $o$  the first activity in an optimal schedule. Since  $g.end \leq o.end$ , replace  $o$  with  $g$ . Every later activity that started after  $o$  ended also starts after  $g$  ends. The schedule remains feasible with the same count. Therefore an optimum begins with  $g$ ; repeat on activities starting after  $g.end$ .

## TRACE IT | Dry run

Activities  $[1,4)$ ,  $[3,5)$ ,  $[0,6)$ ,  $[5,7)$ ,  $[3,9)$ ,  $[5,9)$ ,  $[6,10)$ ,  $[8,11)$ ,  $[8,12)$ ,  $[2,14)$ ,  $[12,16)$  sort by finish. Greedy selects  $[1,4)$ ,  $[5,7)$ ,  $[8,11)$ ,  $[12,16)$ , count 4.

Sorting costs  $O(n \log n)$  and scanning  $O(n)$ . If already sorted by finish, the scan is linear. Returning original IDs requires preserving them in records. Under half-open semantics,  $[t, t)$  is empty: it is compatible with every activity, should always be retained when maximizing count, and must not advance the last occupied end time.

## Counterexamples to tempting rules

- Earliest start picks  $[0,6)$  first and can lose shorter compatible activities.
- Shortest duration can pick an interval positioned in the middle that blocks good choices on both sides.
- Fewest conflicts is not the proven criterion and can be expensive to maintain.

The objective here is maximum count. Weighted interval scheduling, where activities have values, requires dynamic programming in general.

## Pattern 2: merge interval coverage

Sort intervals by start, then end. Keep a current covered interval. If the next interval overlaps under the endpoint contract, extend the current end; otherwise emit the current interval and begin another.

Invariant: emitted intervals are sorted and disjoint and exactly cover all processed input except the current merged component. Because future intervals start no earlier than the current one, once a gap appears the emitted component can never be extended later.

For closed intervals  $[1,3]$ ,  $[2,6]$ ,  $[8,10]$ ,  $[10,12]$ , output is  $[1,6]$ ,  $[8,12]$ ; touching at 10 merges. Under half-open interval semantics,  $[8,10)$  and  $[10,12)$  do not overlap unless product rules explicitly coalesce adjacency.

Time is  $O(n \log n)$ , space depends on sorting/copying and output. This greedy sweep computes the union; it is not the same objective as selecting the maximum number of nonoverlapping activities.

The implementation intentionally appears in both the arrays and greedy volumes. The arrays chapter uses it to teach defensive copying, nested-array ownership, and mutation safety; this chapter uses the same self-contained method to teach the sorted-gap invariant and greedy finalization proof. Keeping both lets each individual PDF remain runnable without importing source from another volume.

## Pattern 3: Jump Game reachability

At index  $i$ ,  $\text{jumps}[i]$  is the maximum forward length. Scan only indexes known reachable and maintain **farthest**, the greatest index reachable using some processed position.

Invariant before index  $i$ : every index up to **farthest** is reachable, and no processed index can reach beyond **farthest**. If  $i > \text{farthest}$ , there is a gap and the end is unreachable. Otherwise update  $\text{farthest} = \max(\text{farthest}, i + \text{jumps}[i])$ .

Use **long** or a saturating comparison for  $i + \text{jumps}[i]$  to avoid overflow when values are untrusted. Reject negative jump lengths under this model.

For  $[2,3,1,1,4]$ , start reach is 0. Index 0 extends to 2; index 1 extends to 4, reaching the end. For  $[3,2,1,0,4]$ , **farthest** becomes 3, but index 4 lies beyond it, so return false.

Time is  $O(n)$ , space  $O(1)$ . The proof is about the frontier of all processed choices, not choosing one specific next index.

## Pattern 4: minimum jumps as BFS layers

When the end is reachable, maintain:

- **currentEnd**: farthest index reachable with the current number of jumps;
- **farthest**: farthest index reachable with one additional jump from positions in the current layer;
- **jumps**: number of completed layers.

Scan indexes before the last. Update **farthest**. When  $i == \text{currentEnd}$ , every position reachable with the current jump count has been considered; increment jumps and set **currentEnd** = **farthest**.

Stays-ahead invariant: after completing a layer, `currentEnd` is the maximum boundary any sequence using that many jumps can reach, because every possible predecessor in the prior reachable interval contributed its reach. Therefore the first layer reaching the end uses the minimum jumps.

For `[2,3,1,1,4]`, the first layer considers index 0 and ends at 2. The second layer considers indexes 1 and 2 and reaches 4. Answer: 2.

Detect a stalled layer (`farthest == currentEnd`) and return unreachable. Time  $O(n)$ , space  $O(1)$ .

### Pattern 5: gas station circuit

At station `i`, gain `gas[i]` and spend `cost[i]` to reach the next station. Two facts produce the greedy solution.

- 1 If total gas is less than total cost, no start can complete the circuit.
- 2 If a tentative start accumulates a negative tank at index `i`, no station from that start through `i` can be a valid start; restart at `i + 1`.

Why can the whole failed segment be skipped? Before each intermediate station, the tentative tank was nonnegative. Starting later removes some nonnegative prefix contribution, so it cannot produce a better tank by the same failure point.

#### TRACE IT | Dry run

`gas=[1,2,3,4,5]`, `cost=[3,4,5,1,2]`. Deltas are `[-2,-2,-2,3,3]`. Starts 0,1,2 fail and reset. Starting at 3 accumulates 3,6, then wraps through negative deltas without dropping below zero. Total delta is zero, so start 3 works.

Time  $O(n)$ , space  $O(1)$ . Use `long` totals. Equal-length, nonempty arrays and nonnegative quantities are reasonable contracts. When multiple starts work, the scan returns one, not necessarily every start.

### Pattern 6: scheduling objectives choose different rules

There is no universal greedy scheduling order.

| Objective/model                                              | Proven order                             |
|--------------------------------------------------------------|------------------------------------------|
| maximize number of compatible activities                     | earliest finish                          |
| minimize maximum lateness on one machine, all jobs available | earliest deadline                        |
| minimize average completion time, unweighted jobs            | shortest processing time                 |
| minimize weighted completion sum                             | ratio/order derived by pairwise exchange |
| minimum rooms for fixed intervals                            | sort starts, track earliest end          |

## Earliest deadline first for maximum lateness

A job has processing time  $p$  and deadline  $d$ . Completion time  $C$  gives lateness  $C - d$ ; the objective minimizes maximum lateness. Sort by nondecreasing deadline.

Exchange proof: if adjacent jobs  $a, b$  are inverted with  $d_a > d_b$ , swap them. Earlier work before the pair is fixed. After swapping,  $b$  completes earlier and  $a$  completes at the original pair completion time. The maximum lateness of the pair does not increase. Repeatedly remove inversions to obtain deadline order.

This does not minimize missed-deadline count, weighted tardiness, or a multi-machine objective. Change the objective and the proof must be rebuilt.

## Pattern 7: Huffman optimal merging

Given symbol frequencies, Huffman coding repeatedly combines the two least frequent subtrees. Their sum becomes a new subtree frequency inserted into a min-heap. Traversing left/right assigns prefix-free code bits; depth times frequency contributes to encoded length.

The merge cost is the sum of every combined frequency. For  $[5, 9, 12, 13, 16, 45]$ , combine  $5+9=14$ ,  $12+13=25$ ,  $14+16=30$ ,  $25+30=55$ ,  $45+55=100$ . Total cost is  $14+25+30+55+100=224$  bits for the corresponding binary tree model.

Exchange idea: in some optimal prefix tree, the two least frequent symbols can be placed as deepest siblings. Combining them reduces the problem to a smaller optimal prefix-code instance. This justifies the repeated local merge.

Time is  $O(n \log n)$ , heap space  $O(n)$ . Ties permit multiple optimal trees; deterministic file formats need a tie-breaking and canonical-code specification. One symbol needs a format policy, often a one-bit code even though tree depth is zero.

The same rule minimizes total merge cost for files/ropes under additive merge cost.

## Pattern 8: fractional knapsack

Items have value and positive weight; fractions are allowed. Sort by decreasing value-to-weight ratio and take as much as possible in that order.

Exchange proof: if a solution uses weight from a lower-ratio item while capacity could instead hold an equal weight from a higher-ratio item, exchanging that weight does not reduce value and usually improves it. Repeating exchanges yields greedy ratio order.

For capacity 50 and items  $(value, weight) = (60, 10), (100, 20), (120, 30)$ , ratios are 6, 5, 4. Take first two fully (weight 30, value 160), then 20/30 of the third (value 80), total 240.

Sorting costs  $O(n \log n)$ ; scan  $O(n)$ . Validate finite nonnegative values and positive weights. Floating-point output tolerance must be defined. The reference comparator interprets each finite `double` through `BigDecimal.valueOf` and compares exact decimal cross-products, avoiding both primitive multiplication overflow and the collapse of distinct huge ratios to `Infinity`. A financial API should accept decimal values directly instead of first accepting binary floating-point inputs.

## Critical counterexample: 0/1 knapsack

If fractions are forbidden, ratio greedy fails on the same items: it takes values 60 and 100 for 160, while items 100 and 120 fit exactly for 220. The 0/1 problem needs dynamic programming or another exact strategy under typical constraints. A tiny wording change invalidates the proof.

## Dijkstra as a greedy boundary

Dijkstra repeatedly settles the unsettled vertex with smallest tentative distance. The cut is settled versus unsettled vertices. With nonnegative edges, any alternate path entering the unsettled side later cannot reduce that minimum: it starts at least as large and adds a nonnegative amount.

A negative edge destroys the cut argument. Example: edges  $s \rightarrow a=2$ ,  $s \rightarrow b=5$ ,  $b \rightarrow a=-10$ . Settling  $a$  at 2 is premature; path  $s \rightarrow b \rightarrow a$  costs -5. Use Bellman-Ford, or topological relaxation for a DAG, when negative edges are allowed.

This illustrates a reusable habit: state the exact assumption used by the greedy proof. If input violates it, fail validation or choose another algorithm.

## Greedy versus dynamic programming

| Question                                                              | Greedy implication                | DP implication                          |
|-----------------------------------------------------------------------|-----------------------------------|-----------------------------------------|
| Can one optimal solution be exchanged to start with the local choice? | promising                         | if no, greedy unproved                  |
| Does one scalar frontier summarize all useful partial solutions?      | promising                         | multiple incomparable states suggest DP |
| Does capacity couple current and future combinations?                 | often dangerous                   | state may include capacity              |
| Are choices fractional/continuous?                                    | ratio exchange may work           | indivisible choices often need DP       |
| Can a small adversarial example beat the ranking?                     | reject exact greedy               | derive recurrence or search             |
| Is approximate output acceptable?                                     | heuristic may be valid with label | exact DP may be too expensive           |

When proposing greedy, actively search for counterexamples: equal scores with different future effects, one large choice versus several medium choices, negative values, and boundary ties. A heuristic may still be useful in production, but document approximation behavior and monitor quality.

## Runnable Java 21 reference implementation

Run with `java -ea GreedyInterviewPatterns`.

JAVA (continued 1/8)

```
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

public final class GreedyInterviewPatterns {
    private GreedyInterviewPatterns() {}

    public record Activity(String id, int start, int end) {
        public Activity {
            if (id == null || start > end) {
                throw new IllegalArgumentException("invalid activity");
            }
        }
    }

    public record Job(String id, int processingTime, int deadline) {
        public Job {
            if (id == null || processingTime < 0) {
                throw new IllegalArgumentException("invalid job");
            }
        }
    }

    public record ScheduleResult(List<String> order, long maximumLateness) {}

    public record FractionalItem(double value, double weight) {
        public FractionalItem {
            if (!Double.isFinite(value) || !Double.isFinite(weight)
                || value < 0 || weight <= 0) {
                throw new IllegalArgumentException("invalid item");
            }
        }
    }
}
```

## JAVA (continued 2/8)

```
    }  
    }  
}  
  
public static List<Activity> selectMaximumActivities(List<Activity> activities) {  
    if (activities == null || activities.stream().anyMatch(java.util.Objects::isNull)) {  
        throw new IllegalArgumentException("activities must be non-null");  
    }  
    List<Activity> sorted = new ArrayList<>(activities);  
    sorted.sort(Comparator.comparingInt(Activity::end)  
        .thenComparingInt(Activity::start)  
        .thenComparing(Activity::id));  
    List<Activity> selected = new ArrayList<>();  
    long lastEnd = Long.MIN_VALUE;  
    for (Activity activity : sorted) {  
        if (activity.start() == activity.end()) {  
            selected.add(activity);  
            continue;  
        }  
        if (activity.start() >= lastEnd) {  
            selected.add(activity);  
            lastEnd = activity.end();  
        }  
    }  
    return List.copyOf(selected);  
}  
  
public static int[][] mergeClosedIntervals(int[][] intervals) {  
    if (intervals == null) {  
        throw new IllegalArgumentException("intervals must not be null");  
    }  
    int[][] copy = new int[intervals.length][2];  
    for (int i = 0; i < intervals.length; i++) {  
        if (intervals[i] == null || intervals[i].length != 2  
            || intervals[i][0] > intervals[i][1]) {  
            continue;  
        }  
        copy[i][0] = intervals[i][0];  
        copy[i][1] = intervals[i][1];  
    }  
    return copy;  
}
```

## JAVA (continued 3/8)

```
        throw new IllegalArgumentException("invalid interval");
    }
    copy[i] = intervals[i].clone();
}
Arrays.sort(copy, Comparator.comparingInt((int[] interval) -> interval[0])
    .thenComparingInt(interval -> interval[1]));
List<int[]> merged = new ArrayList<>();
for (int[] interval : copy) {
    if (merged.isEmpty() || interval[0] > merged.get(merged.size() - 1)[1]) {
        merged.add(interval.clone());
    } else {
        int[] last = merged.get(merged.size() - 1);
        last[1] = Math.max(last[1], interval[1]);
    }
}
return merged.toArray(int[][]::new);
}

public static boolean canReachEnd(int[] jumps) {
    validateJumps(jumps);
    if (jumps.length == 0) {
        return false;
    }
    long farthest = 0;
    for (int i = 0; i < jumps.length && i <= farthest; i++) {
        farthest = Math.max(farthest, (long) i + jumps[i]);
        if (farthest >= jumps.length - 1L) {
            return true;
        }
    }
    return false;
}

public static int minimumJumps(int[] jumps) {
    validateJumps(jumps);
```

## JAVA (continued 4/8)

```
    if (jumps.length <= 1) {
        return jumps.length == 1 ? 0 : -1;
    }
    long currentEnd = 0;
    long farthest = 0;
    int used = 0;
    for (int i = 0; i < jumps.length - 1 && i <= currentEnd; i++) {
        farthest = Math.max(farthest, (long) i + jumps[i]);
        if (i == currentEnd) {
            if (farthest == currentEnd) {
                return -1;
            }
            used++;
            currentEnd = farthest;
            if (currentEnd >= jumps.length - 1) {
                return used;
            }
        }
    }
    return -1;
}

public static int gasStationStart(int[] gas, int[] cost) {
    if (gas == null || cost == null || gas.length == 0 || gas.length != cost.length) {
        throw new IllegalArgumentException("equal nonempty arrays required");
    }
    long total = 0;
    long tank = 0;
    int candidate = 0;
    for (int i = 0; i < gas.length; i++) {
        if (gas[i] < 0 || cost[i] < 0) {
            throw new IllegalArgumentException("gas and cost must be nonnegative");
        }
        long delta = (long) gas[i] - cost[i];
        total += delta;
    }
}
```

## JAVA (continued 5/8)

```
        tank += delta;
        if (tank < 0) {
            candidate = i + 1;
            tank = 0;
        }
    }
    return total >= 0 ? candidate % gas.length : -1;
}

public static ScheduleResult earliestDeadlineSchedule(List<Job> jobs) {
    if (jobs == null || jobs.stream().anyMatch(java.util.Objects::isNull)) {
        throw new IllegalArgumentException("jobs must be non-null");
    }
    List<Job> sorted = new ArrayList<>(jobs);
    sorted.sort(Comparator.comparingInt(Job::deadline).thenComparing(Job::id));
    List<String> order = new ArrayList<>();
    long time = 0;
    long maximumLateness = Long.MIN_VALUE;
    for (Job job : sorted) {
        time = Math.addExact(time, job.processingTime());
        maximumLateness = Math.max(maximumLateness, time - job.deadline());
        order.add(job.id());
    }
    if (sorted.isEmpty()) {
        maximumLateness = 0;
    }
    return new ScheduleResult(List.copyOf(order), maximumLateness);
}

public static long huffmanMergeCost(long[] frequencies) {
    if (frequencies == null) {
        throw new IllegalArgumentException("frequencies must not be null");
    }
    PriorityQueue<Long> queue = new PriorityQueue<>();
    for (long frequency : frequencies) {
```

## JAVA (continued 6/8)

```
        if (frequency <= 0) {
            throw new IllegalArgumentException("frequencies must be positive");
        }
        queue.add(frequency);
    }
    long cost = 0;
    while (queue.size() > 1) {
        long merged = Math.addExact(queue.remove(), queue.remove());
        cost = Math.addExact(cost, merged);
        queue.add(merged);
    }
    return cost;
}

public static double fractionalKnapsack(FractionalItem[] items, double capacity) {
    if (items == null || !Double.isFinite(capacity) || capacity < 0) {
        throw new IllegalArgumentException("invalid items or capacity");
    }
    FractionalItem[] copy = items.clone();
    for (FractionalItem item : copy) {
        if (item == null) {
            throw new IllegalArgumentException("item must not be null");
        }
    }
    Arrays.sort(copy, GreedyInterviewPatterns::compareRatioDescending);
    double remaining = capacity;
    double value = 0;
    for (FractionalItem item : copy) {
        double taken = Math.min(remaining, item.weight());
        value += item.value() * (taken / item.weight());
        remaining -= taken;
        if (remaining == 0.0) {
            break;
        }
    }
}
```

## JAVA (continued 7/8)

```
        return value;
    }

    private static int compareRatioDescending(FractionalItem first,
        FractionalItem second) {
        BigDecimal firstCross = BigDecimal.valueOf(first.value())
            .multiply(BigDecimal.valueOf(second.weight()));
        BigDecimal secondCross = BigDecimal.valueOf(second.value())
            .multiply(BigDecimal.valueOf(first.weight()));
        int ratioOrder = secondCross.compareTo(firstCross);
        if (ratioOrder != 0) {
            return ratioOrder;
        }
        int valueOrder = Double.compare(second.value(), first.value());
        if (valueOrder != 0) {
            return valueOrder;
        }
        return Double.compare(first.weight(), second.weight());
    }

    private static void validateJumps(int[] jumps) {
        if (jumps == null) {
            throw new IllegalArgumentException("jumps must not be null");
        }
        for (int jump : jumps) {
            if (jump < 0) {
                throw new IllegalArgumentException("jumps must be nonnegative");
            }
        }
    }

    public static void main(String[] args) {
        List<Activity> activities = List.of(
            new Activity("A", 1, 4), new Activity("B", 3, 5),
            new Activity("C", 0, 6), new Activity("D", 5, 7),
        );
    }
}
```

## JAVA (continued 8/8)

```

        new Activity("E", 8, 11), new Activity("F", 12, 16));
    assert selectMaximumActivities(activities).stream().map(Activity::id).toList()
        .equals(List.of("A", "D", "E", "F"));
    assert selectMaximumActivities(List.of(
        new Activity("long", 0, 10), new Activity("empty", 5, 5))
        .stream().map(Activity::id).toList().equals(List.of("empty", "long"));
    assert Arrays.deepEquals(mergeClosedIntervals(
        new int[][] {{1, 3}, {2, 6}, {8, 10}, {10, 12}},
        new int[][] {{1, 6}, {8, 12}});

    assert canReachEnd(new int[] {2, 3, 1, 1, 4});
    assert !canReachEnd(new int[] {3, 2, 1, 0, 4});
    assert minimumJumps(new int[] {2, 3, 1, 1, 4}) == 2;
    assert minimumJumps(new int[] {3, 2, 1, 0, 4}) == -1;
    assert gasStationStart(new int[] {1, 2, 3, 4, 5},
        new int[] {3, 4, 5, 1, 2}) == 3;

    ScheduleResult schedule = earliestDeadlineSchedule(List.of(
        new Job("A", 3, 7), new Job("B", 2, 4), new Job("C", 1, 9)));
    assert schedule.order().equals(List.of("B", "A", "C"));
    assert schedule.maximumLateness() == -2;

    assert huffmanMergeCost(new long[] {5, 9, 12, 13, 16, 45}) == 224;
    double value = fractionalKnapsack(new FractionalItem[] {
        new FractionalItem(60, 10), new FractionalItem(100, 20),
        new FractionalItem(120, 30)}, 50);
    assert Math.abs(value - 240.0) < 1e-9;
    double hugeRatio = fractionalKnapsack(new FractionalItem[] {
        new FractionalItem(Double.MAX_VALUE / 2.0, Double.MIN_VALUE),
        new FractionalItem(Double.MAX_VALUE, Double.MIN_VALUE)},
        Double.MIN_VALUE);
    assert hugeRatio == Double.MAX_VALUE;
}
}

```

## WHY IT WORKS | Complexity and proof table

| Pattern                     | Time                                | Space                    | Proof core                            |
|-----------------------------|-------------------------------------|--------------------------|---------------------------------------|
| activity selection          | $O(n \log n)$                       | sorting/output dependent | exchange earliest finish              |
| merge intervals             | $O(n \log n)$                       | copy/output dependent    | sorted gap finalizes coverage         |
| jump reachability/min jumps | $O(n)$                              | $O(1)$                   | frontier stays ahead                  |
| gas station                 | $O(n)$                              | $O(1)$                   | failed segment cannot contain start   |
| earliest-deadline schedule  | $O(n \log n)$                       | $O(n)$ copy/output       | adjacent inversion exchange           |
| Huffman merge               | $O(n \log n)$                       | $O(n)$                   | least frequencies as deepest siblings |
| fractional knapsack         | $O(n \log n)$                       | $O(n)$ copy              | exchange equal weight by ratio        |
| Dijkstra                    | $O((V+E) \log V)$ typical heap form | graph/frontier           | nonnegative cut safety                |

**WATCH OUT | Edge cases and common mistakes**

- 1 **A ranking without a proof.** Sorting by a plausible field does not establish optimality.
- 2 **Objective changed silently.** Maximum activity count differs from maximum value.
- 3 **Endpoint ambiguity.** Define whether touching activities are compatible and touching intervals merge.
- 4 **Jump overflow.** Widen `index + reach` before comparison.
- 5 **Empty Jump Game.** Decide whether no start means false; the toolkit returns false and minimum `-1`.
- 6 **Gas total omitted.** Local restart logic alone cannot make an impossible circuit possible.
- 7 **One scheduling order reused.** Earliest deadline, shortest processing, and earliest finish solve different objectives.
- 8 **Huffman zero/negative frequency.** Remove zero-frequency symbols before construction and reject invalid counts.
- 9 **Nondeterministic Huffman ties.** Canonicalize codes when serialized compatibility matters.
- 10 **Fractional rule applied to 0/1 items.** Indivisibility destroys the exchange.
- 11 **Double ratio assumed exact.** Define numerical tolerance or use suitable exact decimals.
- 12 **Dijkstra with negative weights.** The greedy cut proof fails.
- 13 **Mutation hidden in sorting.** Copy caller inputs or document ownership transfer.
- 14 **Heuristic presented as exact.** Label approximation and quantify quality when proof is absent.

## SDE-2 production follow-ups

- **Tie policy:** deterministic IDs after the primary greedy key make results reproducible without changing the proof when ties are equivalent.
- **Online arrivals:** offline sorted proofs may not survive streaming arrivals. Competitive analysis or reservation rules may replace offline optimality.
- **Cancellation:** schedules change when jobs cancel; maintain data structures and recompute decisions only if irrevocability permits.
- **Fairness:** throughput-optimal priority may starve large or low-priority jobs. Aging and quotas deliberately modify the objective.
- **SLAs:** maximum lateness, missed count, weighted tardiness, and percentile latency are different metrics. Choose one explicitly.
- **Numeric safety:** use `long` for cumulative time, gas, cost, and merge frequencies; use `BigInteger` or bounded validation beyond long range.
- **Input ownership:** sorting is mutation. Copy at API boundaries unless ownership transfer is explicit.
- **Auditing:** retain why each item was accepted, rejected, or delayed when business decisions require explanation.
- **Approximation:** NP-hard scheduling and packing may use greedy approximation. State approximation guarantees when known and monitor gap against small exact samples.
- **Changing priorities:** an object already in a heap must be reinserted or versioned; mutating priority fields violates ordering.

### TRY IT | Exercises with model checkpoints

#### Exercise 1: erase overlapping intervals

Return the minimum intervals to remove so the rest do not overlap.

**Model checkpoints:** maximize retained compatible intervals by earliest finish; answer is `n - selected`; define endpoint compatibility; preserve original IDs if removals must be reported.

#### Exercise 2: partition labels

Partition a string so each symbol appears in at most one part.

**Model checkpoints:** precompute last occurrence; current segment end is farthest last occurrence among scanned symbols; when index reaches end, the segment is forced and can close; specify code-unit/alphabet contract.

### Exercise 3: job sequencing with deadlines and profit

Unit-time jobs earn profit if completed by deadlines.

**Model checkpoints:** sort profits descending and place each job in the latest free slot no later than deadline; exchange intuition preserves earlier slots; DSU can find slots efficiently; reject this rule for arbitrary processing times.

### Exercise 4: canonical Huffman codes

Produce deterministic bit codes from code lengths.

**Model checkpoints:** Huffman tree determines optimal lengths; sort by length then symbol; assign canonical incremented codes; single-symbol policy and maximum length belong to format; decoder validates oversubscribed/incomplete tables.

### Exercise 5: counterexample workshop

Disprove value/weight greedy for 0/1 knapsack and earliest-start greedy for activity selection.

**Model checkpoints:** counterexample must satisfy all input constraints; compare complete feasible outputs; make it minimal enough to explain; identify exactly which exchange step fails.

### Exercise 6: multi-objective scheduler

Design a scheduler balancing deadlines, fairness, and cost.

**Model checkpoints:** no single textbook order optimizes all objectives; define weights or lexicographic priorities; test starvation; consider an exact small-instance optimizer and a measured heuristic at scale; surface trade-offs to stakeholders.

## Interview answer checklist

- ☐ I stated the exact objective and feasibility constraints.
- ☐ I named the local choice and the state retained after it.
- ☐ I supplied exchange, stays-ahead, cut, or induction reasoning.
- ☐ I tested tempting alternative rankings with counterexamples.
- ☐ I defined endpoint, tie, empty, unreachable, and numeric behavior.
- ☐ I did not transfer a fractional proof to indivisible choices.
- ☐ I identified the exact assumption behind Dijkstra or another greedy cut.
- ☐ I can explain when DP or approximation is necessary.
- ☐ I covered determinism, fairness, online arrivals, and ownership.

## RECAP | Summary

Greedy algorithms are local commitments backed by global structure. Earliest finish admits an exchange for activity count. Sorted interval coverage finalizes components at gaps. Jump Game frontiers stay ahead; gas-station failures eliminate whole start segments. Earliest deadline minimizes maximum lateness under its specific model. Huffman repeatedly joins least frequencies, and fractional knapsack exchanges weight toward higher ratios. Dijkstra's cut is safe only with nonnegative edges. Counterexamples define the boundary: weighted intervals, 0/1 capacity, negative edges, and changed scheduling objectives need different reasoning. The SDE-2 standard is proof first, implementation second, and explicit operational trade-offs throughout.

**Part II - Practice and Handoff**

## Practice Ladder and Next Steps

**TRY IT | Practice ladder**

- Foundation: activity selection and interval merges
- Interview Core: jumps, gas station, and scheduling
- SDE-2 Follow-up: construct or refute proofs
- Challenge: compare greedy, DP, and graph formulations

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

### Navigation

[Previous volume: 15 - Graphs: Traversal, Ordering, Paths, and Union-Find](#)

[Series index](#)

[Next volume: 17 - Dynamic Programming: State, Transitions, and Optimization](#)

**TRY IT | Completion check**

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

# Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

| STEP | FOCUSED LEARNING PATH                                         |
|------|---------------------------------------------------------------|
| 1    | Java Foundations for Problem Solving                          |
| 2    | Time and Space Complexity                                     |
| 3    | Number Systems and Math Foundations for DSA Interviews        |
| 4    | Bit Manipulation in Java                                      |
| 5    | Loop Mastery, Patterns, and Index Calculations                |
| 6    | Arrays and Array Problem-Solving Patterns                     |
| 7    | Strings and String Problem-Solving Patterns                   |
| 8    | Hashing: Maps, Sets, Frequency, and Prefix State              |
| 9    | Recursion and Backtracking in Java                            |
| 10   | Linked Lists: Pointer Reasoning and Mutation                  |
| 11   | Stacks, Queues, Deques, and Monotonic Patterns                |
| 12   | Binary Search: Bounds, Answers, and Invariants                |
| 13   | Trees, Binary Search Trees, and Tries                         |
| 14   | Heaps, Priority Queues, Selection, and Top-K                  |
| 15   | Graphs: Traversal, Ordering, Paths, and Union-Find            |
| 16   | Greedy Algorithms: Recognition, Proofs, and Scheduling        |
| 17   | Dynamic Programming: State, Transitions, and Optimization     |
| 18   | Advanced Java Engineering and the SDE-2 Interview (Parts A-J) |

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

# About the Author

## Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

## Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

## Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

## Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

**Connect:** [LinkedIn](#) | [GitHub](#)

# Copyright and Publishing Notes

---

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 16 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.